

TypeScript fundamentals answer key

Section A: Basic types and type decisions

Exercise 1: Catch an incorrect value before the code runs

```
function calculateTicketCost(  
  price: number,  
  quantity: number  
): number {  
  return price * quantity;  
}  
  
console.log(calculateTicketCost(250, 3));  
  
// TypeScript rejects this call:  
// console.log(calculateTicketCost(250, "three"));
```

Expected output:

750

TypeScript detects the incorrect argument while checking the code. The program does not need to run before the mismatch is found.

Without the parameter types, JavaScript would attempt to use `"three"` in the multiplication. The result would be `NaN`, which would only be discovered while the program was running or when someone noticed the incorrect output.

Exercise 2: Use inference where it is sufficient

Before adding the status restriction, TypeScript infers:

```
const courseTitle = "TypeScript fundamentals";  
// "TypeScript fundamentals"  
  
let seatsRemaining = 18;  
// number  
  
let registrationOpen = true;  
// boolean
```

```
let registrationStatus = "open";  
// string
```

These assignments are accepted:

```
seatsRemaining = 12;  
registrationOpen = false;  
registrationStatus = "closed";
```

This assignment is rejected:

```
// seatsRemaining = "12";
```

`seatsRemaining` is a number, while `"12"` is a string.

The registration status can be restricted with a literal union:

```
type RegistrationStatus =  
  | "open"  
  | "full"  
  | "cancelled";  
  
const courseTitle = "TypeScript fundamentals";  
let seatsRemaining = 18;  
let registrationOpen = true;  
  
let registrationStatus: RegistrationStatus = "open";  
  
registrationStatus = "full";  
registrationStatus = "cancelled";  
  
// Invalid:  
// registrationStatus = "closed";
```

The other variables do not need explicit annotations because TypeScript already infers useful types for them.

Exercise 3: Choose a suitable type for each value

```
const sessionTopics: string[] = [  
  "Functions",  
  "Objects",  
  "TypeScript",  
];
```

```
const sessionTime: [string, string] = [
  "10:00",
  "13:00",
];

let latestAssessmentResult:
  | number
  | "not attempted" = "not attempted";

latestAssessmentResult = 84;
```

`sessionTopics` is an array because it may contain any number of topic names.

`sessionTime` is a tuple because it always contains two values in a fixed order.

`latestAssessmentResult` is a union because it may contain either a number or one specific string value.

Exercise 4: Restrict the values stored in an array

```
type ScoreEntry = number | "absent";

const scores: ScoreEntry[] = [
  78,
  84,
  "absent",
  91,
];

scores.push(88);
scores.push("absent");

// Invalid:
// scores.push("late");
```

Using `number | string` would be too broad because it would allow any string. The literal `"absent"` limits the accepted string value to the one required by the application.

Exercise 5: Work safely with a union type

```
function formatMemberId(
  id: string | number
): string {
  if (typeof id === "number") {
    return `MEM-${id}`;
  }
```

```

    }

    return id.toUpperCase();
}

console.log(formatMemberId(204));
console.log(formatMemberId("ext-17"));

```

Expected output:

```

MEM-204
EXT-17

```

Before the `typeof` check, `id` may be either a string or a number.

Inside the first branch, TypeScript knows that `id` is a number. After that branch, TypeScript knows that the remaining possibility is a string.

Exercise 6: Create a typed package scan log

```

type PackageWeight =
  | number
  | "unknown";

type ScanResult =
  | "accepted"
  | "held"
  | "rejected";

type ScanRecord = [
  string,
  PackageWeight,
  ScanResult
];

const scanLog: ScanRecord[] = [
  ["PKG-101", 2.4, "accepted"],
  ["PKG-102", "unknown", "held"],
  ["PKG-103", 7.8, "rejected"],
];

scanLog.push([
  "PKG-104",
  3.2,
  "accepted",
]);

```

```
// Invalid:
// scanLog.push([
//   "PKG-105",
//   4.1,
//   "delivered",
// ]);
```

Each record is a tuple because the three positions have fixed meanings. The scan log is an array because it can contain several records.

Section B: Objects, interfaces, and reusable shapes

Exercise 7: Describe nested service request data

```
type Priority =
  | "low"
  | "medium"
  | "high";

interface Resident {
  name: string;
  apartment: string;
}

interface ServiceRequest {
  requestId: string;
  issue: string;
  priority: Priority;
  resident: Resident;
}

const request: ServiceRequest = {
  requestId: "SR-104",
  issue: "Water leakage",
  priority: "high",
  resident: {
    name: "Anita Rao",
    apartment: "B-204",
  },
};
```

`Resident` is defined separately so the same structure can be reused in other requests or parts of the application.

The `Priority` union prevents values outside the three supported priority levels.

Exercise 8: Use a type alias and an interface for different purposes

```
type TicketStatus =  
  | "open"  
  | "in-progress"  
  | "resolved";  
  
interface SupportTicket {  
  readonly id: string;  
  subject: string;  
  status: TicketStatus;  
  assignedAgent?: string;  
}  
  
const unassignedTicket: SupportTicket = {  
  id: "T-101",  
  subject: "Unable to reset password",  
  status: "open",  
};  
  
const assignedTicket: SupportTicket = {  
  id: "T-102",  
  subject: "Payment failed",  
  status: "in-progress",  
  assignedAgent: "Maya",  
};
```

`TicketStatus` is a type alias because it represents a union of possible values.

`SupportTicket` is an interface because it describes the properties expected in a ticket object.

Both interfaces and type aliases can describe object shapes. The type alias is required here for the literal union.

Exercise 9: Use an optional property safely

```
interface Attendee {  
  name: string;  
  phone?: string;  
}  
  
const attendee: Attendee = {  
  name: "Kabir",  
};  
  
console.log(
```

```
attendee.phone?.toUpperCase()  
  ?? "Phone number not provided"  
);
```

Expected output:

```
Phone number not provided
```

Optional chaining stops the method call when `phone` is absent.

The nullish coalescing operator provides the fallback message when the result is `undefined`.

An equivalent solution using an `if` statement is also correct:

```
if (attendee.phone) {  
  console.log(  
    attendee.phone.toUpperCase()  
  );  
} else {  
  console.log(  
    "Phone number not provided"  
  );  
}
```

Exercise 10: Compare `const` and `readonly`

This change is accepted:

```
registration.learnerName = "Riya";
```

`learnerName` is not readonly, so the property may be updated.

This change is rejected:

```
// registration.registrationId =  
//   "REG-101";
```

`registrationId` is marked as readonly. It cannot be assigned a new value through the `CourseRegistration` type.

This change is also rejected:

```
// registration = {  
//   registrationId: "REG-102",  
//   learnerName: "Farhan",  
//   status: "active",  
// };
```

The variable was declared with `const`, so it cannot refer to a different object.

The two restrictions are different:

- `const` prevents the variable from being reassigned.
- `readonly` prevents a specific object property from being reassigned.

Declaring an object with `const` does not make all its properties readonly. Mutable properties on the same object can still be updated.

Exercise 11: Extend and combine existing types

```
interface UserProfile {  
  readonly id: number;  
  name: string;  
  email: string;  
}  
  
interface MentorProfile  
  extends UserProfile {  
    expertise: string[];  
    yearsOfExperience: number;  
  }  
  
type Availability = {  
  availableDays: string[];  
  acceptsNewLearners: boolean;  
};  
  
type AvailableMentor =  
  MentorProfile & Availability;  
  
const mentor: AvailableMentor = {  
  id: 42,  
  name: "Leena",  
  email: "leena@example.com",  
  expertise: [  
    "JavaScript",  
    "TypeScript",  
  ],  
  yearsOfExperience: 7,  
  availableDays: [  

```

```

        "Saturday",
        "Sunday",
    ],
    acceptsNewLearners: true,
};

```

`MentorProfile` receives `id`, `name`, and `email` from `UserProfile`. It adds `expertise` and `yearsOfExperience`.

The intersection creates a type that must satisfy both `MentorProfile` and `Availability`.

Exercise 12: Model workshop registration data

```

type WorkshopMode =
  | "online"
  | "in-person";

type WorkshopStatus =
  | "scheduled"
  | "completed"
  | "cancelled";

interface Venue {
  building: string;
  room: string | number;
}

interface Workshop {
  readonly id: number;
  title: string;
  mode: WorkshopMode;
  status: WorkshopStatus;
  venue?: Venue;
}

interface Participant {
  readonly id: number;
  name: string;
  dietaryPreference?: string;
}

interface WorkshopRegistration {
  readonly registrationCode: string;
  workshop: Workshop;
  participant: Participant;
  registrationDate: string;
}

```

Online registration:

```
const onlineRegistration:
  WorkshopRegistration = {
    registrationCode: "WR-101",
    workshop: {
      id: 1,
      title: "TypeScript basics",
      mode: "online",
      status: "scheduled",
    },
    participant: {
      id: 201,
      name: "Mohan",
    },
    registrationDate: "2026-08-03",
  };
```

In-person registration:

```
const inPersonRegistration:
  WorkshopRegistration = {
    registrationCode: "WR-102",
    workshop: {
      id: 2,
      title: "DOM debugging lab",
      mode: "in-person",
      status: "scheduled",
      venue: {
        building: "Learning Centre",
        room: "Lab 2",
      },
    },
    participant: {
      id: 202,
      name: "Sara",
      dietaryPreference: "Vegetarian",
    },
    registrationDate: "2026-08-03",
  };
```

The optional properties allow online workshops to omit a venue and participants to omit a dietary preference when neither value is required.

Section C: Functions, classes, and generics

Exercise 13: Calculate the total value of an order

```
interface CartItem {
  name: string;
  price: number;
  quantity: number;
}

function calculateOrderTotal(
  items: CartItem[]
): number {
  return items.reduce(
    (total, item) => {
      return total
        + item.price * item.quantity;
    },
    0
  );
}

const orderItems: CartItem[] = [
  {
    name: "Notebook",
    price: 120,
    quantity: 2,
  },
  {
    name: "Pen set",
    price: 80,
    quantity: 1,
  },
];

console.log(
  calculateOrderTotal(orderItems)
);
```

Expected output:

320

The parameter type requires every item to contain the properties defined by `CartItem`.

The return type requires the function to return a number.

Exercise 14: Type an arrow function with a default parameter

```
interface Session {
  title: string;
  durationMinutes: number;
}

const getLongSessions = (
  sessions: Session[],
  minimumDuration = 60
): Session[] => {
  return sessions.filter(
    (session) => {
      return session.durationMinutes
        >= minimumDuration;
    }
  );
};

const sessions: Session[] = [
  {
    title: "Type annotations",
    durationMinutes: 45,
  },
  {
    title: "Classes",
    durationMinutes: 75,
  },
];

console.log(
  getLongSessions(sessions)
);

console.log(
  getLongSessions(sessions, 40)
);
```

The first call returns:

```
[
  {
    title: "Classes",
    durationMinutes: 75,
  },
]
```

The second call returns both sessions because both durations are at least `40`.

TypeScript infers `minimumDuration` as a number from its default value.

Exercise 15: Handle an optional parameter

```
function formatInvoice(  
  total: number,  
  currency = "INR",  
  discountPercent?: number  
): string {  
  const discount =  
    discountPercent ?? 0;  
  
  const discountedTotal =  
    total - (total * discount) / 100;  
  
  return (  
    `${currency} ` +  
    discountedTotal.toFixed(2)  
  );  
}  
  
console.log(formatInvoice(1000));  
  
console.log(  
  formatInvoice(1000, "INR", 10)  
);  
  
console.log(  
  formatInvoice(1000, "USD")  
);
```

Expected output:

```
INR 1000.00  
INR 900.00  
USD 1000.00
```

`currency` always has a string value because it has a default.

`discountPercent` may be `undefined`. The nullish coalescing operator replaces `undefined` with `0` before the calculation.

Exercise 16: Replace repeated objects with a class

```
class HotelRoom {
  readonly roomNumber: number;
  guestName?: string;

  constructor(roomNumber: number) {
    this.roomNumber = roomNumber;
  }

  checkIn(name: string): void {
    this.guestName = name;
  }

  checkOut(): void {
    this.guestName = undefined;
  }

  getStatus(): string {
    if (this.guestName) {
      return (
        `Occupied by ${this.guestName}`
      );
    }

    return "Available";
  }
}

const firstRoom =
  new HotelRoom(101);

const secondRoom =
  new HotelRoom(102);

firstRoom.checkIn("Dev");

console.log(
  firstRoom.getStatus()
);

console.log(
  secondRoom.getStatus()
);
```

Expected output:

Occupied by Dev
Available

Answers to the concept questions:

1. `HotelRoom` is the class.
2. `firstRoom` and `secondRoom` are instances.
3. `new` creates a new instance and runs the class constructor.
4. Inside `checkIn`, `this` refers to the instance on which the method was called.
5. Each instance stores its own property values. Changing `firstRoom.guestName` does not change `secondRoom.guestName`.

Exercise 17: Control access to an account balance

```
class BankAccount {
  public readonly accountNumber:
    string;

  private balance: number;

  constructor(
    accountNumber: string,
    openingBalance = 0
  ) {
    this.accountNumber =
      accountNumber;

    this.balance =
      openingBalance;
  }

  deposit(amount: number): void {
    if (amount > 0) {
      this.balance += amount;
    }
  }

  withdraw(amount: number):
    boolean {
    if (
      amount <= 0
      || amount > this.balance
    ) {
      return false;
    }

    this.balance -= amount;
  }
}
```

```

        return true;
    }

    getBalance(): number {
        return this.balance;
    }
}

const account =
    new BankAccount(
        "AC-501",
        1000
    );

account.deposit(500);

console.log(
    account.withdraw(200)
);

console.log(
    account.getBalance()
);

```

Expected output:

```

true
1300

```

Invalid outside access:

```

// The property is readonly:
// account.accountNumber = "AC-502";

// The property is private:
// console.log(account.balance);

// The property is private:
// account.balance = 5000;

```

Outside code can read `accountNumber` because it is public. It cannot assign a new value because the property is readonly.

Outside code cannot access `balance` directly because it is private. It must use the public methods provided by the class.

Exercise 18: Read a protected property through a subclass

This line is accepted:

```
console.log(  
  lead.getDepartment()  
);
```

Expected output:

```
Engineering
```

This line is rejected:

```
// console.log(lead.department);
```

`TeamLead` extends `StaffMember`, so code inside `TeamLead` may access the protected `department` property.

Outside code cannot access the property directly.

If `department` were private, `TeamLead` would not be allowed to access it either:

```
class StaffMember {  
  private department: string;  
  
  constructor(department: string) {  
    this.department = department;  
  }  
}  
  
class TeamLead extends StaffMember {  
  getDepartment(): string {  
    // Invalid:  
    // return this.department;  
  
    return "Unavailable";  
  }  
}
```

A protected property is accessible inside its declaring class and subclasses. A private property is accessible only inside its declaring class.

Exercise 19: Preserve the item type with a generic function

```
function getFirst<T>(  
  items: T[]  
): T | undefined {  
  return items[0];  
}  
  
const firstTopic = getFirst([  
  "Types",  
  "Interfaces",  
]);  
  
const firstScore = getFirst([  
  78,  
  84,  
  91,  
]);  
  
const firstEmptyValue =  
  getFirst<number>([]);
```

The inferred types are:

```
// string | undefined  
firstTopic;  
  
// number | undefined  
firstScore;  
  
// number | undefined  
firstEmptyValue;
```

The generic type parameter preserves the relationship between the array item type and the return type.

With `any`, the return value loses its original type information. TypeScript would allow unsupported operations on the result without useful checking.

The `undefined` possibility is included because an empty array has no first item.

Exercise 20: Create a reusable generic result type

```
interface Result<T> {  
  success: boolean;  
  message: string;  
  data?: T;  
}
```

```

}

interface CourseSummary {
  id: number;
  title: string;
}

interface LearnerSummary {
  id: number;
  name: string;
}

```

Successful course result:

```

const successfulCourseResult:
  Result<CourseSummary> = {
  success: true,
  message: "Course loaded",
  data: {
    id: 15,
    title: "TypeScript fundamentals",
  },
};

```

Failed course result:

```

const failedCourseResult:
  Result<CourseSummary> = {
  success: false,
  message:
    "Course could not be loaded",
};

```

Successful learner result:

```

const successfulLearnerResult:
  Result<LearnerSummary> = {
  success: true,
  message: "Learner loaded",
  data: {
    id: 9,
    name: "Ishaan",
  },
};

```

The outer structure remains the same in each result:

```
success
message
data
```

The type supplied for `T` controls the type accepted by `data`.

For example:

```
Result<CourseSummary>
Result<LearnerSummary>
```

The first requires course data. The second requires learner data.

Final exercise: Build a small course enrolment register

Part 1: Model the data

```
type CourseMode =
  | "online"
  | "in-person";

type EnrolmentStatus =
  | "active"
  | "cancelled";

interface Course {
  readonly id: number;
  title: string;
  mode: CourseMode;
  capacity: number;
  room?: string;
}

interface Learner {
  readonly id: number;
  name: string;
  email?: string;
}

interface Enrolment {
  readonly enrolmentCode: string;
  course: Course;
  learner: Learner;
  status: EnrolmentStatus;
}
```

```
interface OperationResult<T> {
    success: boolean;
    message: string;
    data?: T;
}
```

Part 2: Describe the course location

```
function getCourseLocation(
    course: Course
): string {
    if (course.mode === "online") {
        return "Online";
    }

    return (
        course.room
        ?? "Room not assigned"
    );
}
```

The function first checks whether the course is online.

For an in-person course, it returns the room when one exists. Otherwise, it returns the fallback message.

Part 3: Register learners

```
class EnrolmentRegister {
    public readonly course: Course;

    private enrolments:
        Enrolment[] = [];

    constructor(course: Course) {
        this.course = course;
    }

    enrol(
        learner: Learner
    ): OperationResult<Enrolment> {
        if (
            this.enrolments.length
            >= this.course.capacity
        ) {
            return {
                success: false,
            };
        }
    }
}
```

```

        message:
            "The course is full",
    };
}

const enrolment: Enrolment = {
    enrolmentCode:
        `ENR-${
            this.enrolments.length + 1
        }`,
    course: this.course,
    learner,
    status: "active",
};

this.enrolments.push(
    enrolment
);

return {
    success: true,
    message:
        "Enrolment created",
    data: enrolment,
};
}

getActiveEnrolments():
    Enrolment[] {
    return this.enrolments.filter(
        (enrolment) => {
            return (
                enrolment.status
                === "active"
            );
        }
    );
}
}

```

The private array can only be accessed through the class methods.

The `enrol` method checks capacity before creating a new enrolment.

The return type connects the result data to the `Enrolment` interface.

Part 4: Test the program

```
const course: Course = {
  id: 301,
  title:
    "TypeScript practice lab",
  mode: "in-person",
  capacity: 2,
  room: "Lab 4",
};

const learnerOne: Learner = {
  id: 1,
  name: "Aman",
  email: "aman@example.com",
};

const learnerTwo: Learner = {
  id: 2,
  name: "Jaya",
};

const learnerThree: Learner = {
  id: 3,
  name: "Omar",
};

const register =
  new EnrolmentRegister(course);

const firstResult =
  register.enrol(learnerOne);

const secondResult =
  register.enrol(learnerTwo);

const thirdResult =
  register.enrol(learnerThree);

console.log(
  getCourseLocation(course)
);

console.log(
  firstResult.message
);

console.log(
  secondResult.message
);
```

```
console.log(  
  thirdResult.message  
);  
  
console.log(  
  register  
    .getActiveEnrolments()  
    .length  
);
```

Expected output:

```
Lab 4  
Enrolment created  
Enrolment created  
The course is full  
2
```

The first two learners are enrolled because the course capacity is two.

The third attempt returns an unsuccessful result and does not add another enrolment.